python_dsa_cheatsheet

Python DSA Cheat Sheet — C++ to Python

> For competitive programmers migrating from C++ STL to Python. Structure = equivalent, not identical.

---

0. Quick Mindset Shifts

C++	Python
`int`, `long long`	`int` (arbitrary precision, no overflow)
`INT_MAX / INT_MIN`	`float('inf')` / `float('-inf')`
`NULL`	`None`
`cout << x`	`print(x)`
`cin >> x`	`x = int(input())`
`auto`	just assign it
`typedef`	just use the type directly
Pass by reference `&`	Objects are refs; use `[:]` to copy lists

---

1. Array / Vector → `list`

```
# Declaration
arr = []                      # empty (like vector<int>)
arr = [0] * n                 # size n, filled with 0
arr = [0] * n * m             # flat 1D representation
grid = [[0]*m for _ in range(n)]  # 2D grid (DO NOT use [[0]*m]*n — shallow copy bug)

# Common ops
arr.append(x)          # push_back
arr.pop()              # pop_back — O(1)
arr.pop(i)             # erase at index — O(n)
arr.insert(i, x)       # insert at index — O(n)
arr[i]                 # access — O(1)
len(arr)               # size()
arr.sort()             # sort in-place — O(n log n)
arr.sort(reverse=True) # sort descending
sorted(arr)            # returns new sorted list
arr.reverse()          # reverse in-place
arr[::-1]              # reversed copy
arr.index(x)           # first occurrence of x (raises if not found)
x in arr               # O(n) search
arr.count(x)           # count occurrences

# Slicing
arr[l:r]               # subarray [l, r)  — O(k)
arr[l:r:step]          # with step

# Useful tricks
min(arr), max(arr)
sum(arr)
arr1 + arr2            # concatenate
arr * k                # repeat

# 2D traversal
for i in range(n):
    for j in range(m):
        print(grid[i][j])
```

2. String

```
s = "hello"
s[i]                    # character access
len(s)                  # length
s + t                   # concatenation (creates new string — strings are immutable)
s * k                   # repeat
s[::-1]                 # reverse
s.lower(), s.upper()
s.strip()               # trim whitespace
s.split()               # split by whitespace → list
s.split(',')            # split by delimiter
','.join(arr)           # join list into string
s.replace('a', 'b')
s.find('sub')           # first index, -1 if not found
s.count('a')
s.startswith('he'), s.endswith('lo')
s.isdigit(), s.isalpha(), s.isalnum()
ord('a')                # char to ASCII (like (int)'a')
chr(97)                 # ASCII to char (like (char)97)

# Mutable string — use list
chars = list(s)
chars[0] = 'H'
s = ''.join(chars)

# Multiline / f-string
f"value is {x}"         # like to_string or printf
```

---

3. Stack → `list` (or `deque`)

```
# Use list as stack
stack = []
stack.append(x)         # push — O(1)
stack.pop()             # pop — O(1)
stack[-1]               # top/peek — O(1)
len(stack) == 0         # isEmpty

# Monotonic stack pattern
stack = []
for x in arr:
    while stack and stack[-1] >= x:
        stack.pop()
    stack.append(x)
```

---

4. Queue → `collections.deque`

```
from collections import deque

q = deque()
q.append(x)             # enqueue (push_back) — O(1)
q.popleft()             # dequeue (pop_front) — O(1)
q[0]                    # front — O(1)
q[-1]                   # back — O(1)
len(q) == 0             # isEmpty

# Also supports stack ops:
q.appendleft(x)         # push_front
q.pop()                 # pop_back
```

```python
# BFS template
from collections import deque
def bfs(graph, start):
    visited = set()
    q = deque([start])
    visited.add(start)
    while q:
        node = q.popleft()
        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei)
                q.append(nei)
```

---

5. Priority Queue / Heap → `heapq`

```python
import heapq

# MIN-HEAP (default)
heap = []
heapq.heappush(heap, x)        # push — O(log n)
heapq.heappop(heap)            # pop min — O(log n)
heap[0]                        # peek min — O(1)
heapq.heapify(arr)             # build heap from list — O(n)

# MAX-HEAP trick: negate values
heapq.heappush(heap, -x)
-heapq.heappop(heap)           # returns max

# Heap with tuples (sorts by first element)
heapq.heappush(heap, (priority, value))

# nlargest / nsmallest
heapq.nlargest(k, arr)         # top k largest — O(n log k)
heapq.nsmallest(k, arr)        # top k smallest

# Dijkstra template
import heapq
def dijkstra(graph, src, n):
    dist = [float('inf')] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

---

6. Hash Map → `dict`

```python
# Declaration
mp = {}
mp = dict()

# Operations — all O(1) average
mp[key] = value        # insert / update
mp[key]                # access (KeyError if missing)
mp.get(key, default)   # safe access with default
key in mp              # contains key
del mp[key]            # erase
mp.pop(key, None)      # erase with optional default
```

```python
len(mp)                 # size

# Iteration
for key in mp:              # keys
for key, val in mp.items():# key-value pairs
for val in mp.values():     # values

# Useful patterns
mp[key] = mp.get(key, 0) + 1   # frequency count

# defaultdict — auto-initializes missing keys
from collections import defaultdict
mp = defaultdict(int)          # default 0
mp = defaultdict(list)         # default []
mp = defaultdict(set)
mp[key] += 1                   # no KeyError

# Counter — frequency map from iterable
from collections import Counter
cnt = Counter(arr)             # {'a': 3, 'b': 2, ...}
cnt.most_common(k)             # top k frequent
cnt[x]                         # 0 if missing (no KeyError)
```

---

7. Hash Set → `set`

```python
s = set()
s = {1, 2, 3}          # from literal
s = set(arr)           # from list — O(n)

s.add(x)               # insert — O(1)
s.remove(x)            # erase (KeyError if missing) — O(1)
s.discard(x)           # erase safely — O(1)
x in s                 # O(1) lookup
len(s)

# Set operations
a | b                  # union
a & b                  # intersection
a - b                  # difference (a minus b)
a ^ b                  # symmetric difference
a.issubset(b)
a.issuperset(b)

# Frozen (hashable) set — usable as dict key
fs = frozenset([1, 2, 3])
```

---

8. Ordered Map → `sortedcontainers.SortedList / SortedDict`

> C++ `map` / `set` (red-black tree) → Python's built-in dict is unordered.
> Use `sortedcontainers` for O(log n) ordered ops.

```python
from sortedcontainers import SortedList, SortedDict, SortedSet

sl = SortedList()
sl.add(x)              # O(log n)
sl.remove(x)           # O(log n)
sl[0]                  # min
sl[-1]                 # max
sl.bisect_left(x)      # lower_bound index
sl.bisect_right(x)     # upper_bound index
sl.irange(lo, hi)      # range query iterator
```

```
sd = SortedDict()
sd[key] = val
sd.peekitem(0)          # (min_key, val)
sd.peekitem(-1)         # (max_key, val)
```

---

9. Binary Search → `bisect`

```
import bisect

# Array must be sorted
bisect.bisect_left(arr, x)     # lower_bound: first index where arr[i] ⩾ x
bisect.bisect_right(arr, x)    # upper_bound: first index where arr[i] > x

# Insert and keep sorted
bisect.insort_left(arr, x)
bisect.insort_right(arr, x)

# Manual binary search template
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

# Search on answer template
def feasible(x): ...           # predicate function

lo, hi = 0, max_val
while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid):
        hi = mid
    else:
        lo = mid + 1
# lo is the answer
```

---

10. Linked List (manual)

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# Build: 1 → 2 → 3
head = ListNode(1, ListNode(2, ListNode(3)))

# Traverse
cur = head
while cur:
    print(cur.val)
    cur = cur.next

# Reverse in-place
def reverse(head):
    prev = None
    cur = head
    while cur:
```

```
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev

# Fast & Slow pointer (cycle / midpoint)
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
# slow is at midpoint
```

---

11. Tree (Binary Tree)

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# DFS traversals (recursive)
def inorder(root):
    if not root: return
    inorder(root.left)
    print(root.val)
    inorder(root.right)

def preorder(root):
    if not root: return
    print(root.val)
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if not root: return
    postorder(root.left)
    postorder(root.right)
    print(root.val)

# BFS (level order)
from collections import deque
def level_order(root):
    if not root: return
    q = deque([root])
    while q:
        node = q.popleft()
        print(node.val)
        if node.left:  q.append(node.left)
        if node.right: q.append(node.right)

# Height
def height(root):
    if not root: return 0
    return 1 + max(height(root.left), height(root.right))
```

---

12. Graph Representations

```
# Adjacency list (most common)
n = 5
graph = [[] for _ in range(n)]       # list of lists
graph[u].append(v)                   # directed edge u→v
graph[u].append(v); graph[v].append(u)  # undirected
```

```python
# With weights
graph[u].append((v, w))

# Using defaultdict
from collections import defaultdict
graph = defaultdict(list)
graph[u].append(v)

# Adjacency matrix (dense graphs)
mat = [[0] * n for _ in range(n)]
mat[u][v] = 1

# DFS on graph
def dfs(node, visited, graph):
    visited.add(node)
    for nei in graph[node]:
        if nei not in visited:
            dfs(nei, visited, graph)

# DFS iterative
def dfs_iter(start, graph):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited: continue
        visited.add(node)
        for nei in graph[node]:
            stack.append(nei)
```

---

13. Union-Find (Disjoint Set Union)

```python
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)

# Usage
dsu = DSU(n)
dsu.union(0, 1)
dsu.connected(0, 2)  # False
```

---

14. Segment Tree

```python
class SegTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self.build(arr, 0, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
        else:
            mid = (start + end) // 2
            self.build(arr, 2*node+1, start, mid)
            self.build(arr, 2*node+2, mid+1, end)
            self.tree[node] = self.tree[2*node+1] + self.tree[2*node+2]  # sum tree

    def update(self, node, start, end, idx, val):
        if start == end:
            self.tree[node] = val
        else:
            mid = (start + end) // 2
            if idx <= mid:
                self.update(2*node+1, start, mid, idx, val)
            else:
                self.update(2*node+2, mid+1, end, idx, val)
            self.tree[node] = self.tree[2*node+1] + self.tree[2*node+2]

    def query(self, node, start, end, l, r):
        if r < start or end < l:
            return 0
        if l <= start and end <= r:
            return self.tree[node]
        mid = (start + end) // 2
        return (self.query(2*node+1, start, mid, l, r) +
                self.query(2*node+2, mid+1, end, l, r))

# Usage
st = SegTree([1, 3, 5, 7, 9])
st.query(0, 0, st.n-1, 1, 3)      # sum of range [1,3]
st.update(0, 0, st.n-1, 2, 10)    # update index 2 to 10
```

---

15. Trie (Prefix Tree)

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        cur = self.root
        for c in word:
            if c not in cur.children:
                cur.children[c] = TrieNode()
            cur = cur.children[c]
        cur.end = True

    def search(self, word):
        cur = self.root
        for c in word:
            if c not in cur.children: return False
            cur = cur.children[c]
        return cur.end
```

```
    def startsWith(self, prefix):
        cur = self.root
        for c in prefix:
            if c not in cur.children: return False
            cur = cur.children[c]
        return True
```

---

16. Useful Built-ins & `itertools` / `functools`

```
# Math
abs(x)
pow(x, y, mod)          # modular exponentiation — fast
divmod(a, b)            # returns (quotient, remainder)
math.gcd(a, b)          # gcd
math.lcm(a, b)          # lcm (Python 3.9+)
math.isqrt(n)           # integer sqrt
math.ceil(x), math.floor(x)
math.log(x), math.log2(x), math.log10(x)
math.inf                 # float('inf')

import math

# Functional
from functools import lru_cache
@lru_cache(maxsize=None)   # memoization decorator (replaces manual dp memo dict)
def dp(i, j):
    ...

from functools import reduce
reduce(lambda a, b: a + b, arr)   # fold/accumulate

# Itertools
from itertools import permutations, combinations, combinations_with_replacement, product, accumulate

list(permutations([1,2,3]))         # all permutations
list(combinations([1,2,3], 2))      # nCr combinations
list(product([0,1], repeat=3))      # cartesian product (like nested loops)
list(accumulate([1,2,3,4]))         # prefix sum: [1, 3, 6, 10]
list(accumulate([1,2,3,4], max))    # prefix max

# Sorting with key
arr.sort(key=lambda x: x[1])           # sort by second element
arr.sort(key=lambda x: (-x[0], x[1]))  # multi-key sort

# zip / enumerate
for i, x in enumerate(arr):            # index + value
for a, b in zip(arr1, arr2):           # parallel iteration
list(zip(*matrix))                     # transpose matrix

# any / all
any(x > 5 for x in arr)
all(x > 0 for x in arr)

# map / filter
list(map(int, input().split()))         # read int array
list(filter(lambda x: x > 0, arr))
```

---

17. Input / Output (Competitive Style)

```
import sys
input = sys.stdin.readline             # faster input
```

```python
# Read single int
n = int(input())

# Read multiple ints on one line
a, b, c = map(int, input().split())

# Read array
arr = list(map(int, input().split()))

# Read n lines
for _ in range(n):
    x = int(input())

# Read grid
grid = [list(map(int, input().split())) for _ in range(n)]

# Read string
s = input().strip()

# Fast print
import sys
print = sys.stdout.write              # use print(str(x) + '\n') then
sys.stdout.write(str(ans) + '\n')     # slightly faster for many outputs
```

---

18. Common Patterns & Gotchas

```python
# Integer division
a // b                # floor division (like a/b in C++ for positive ints)
a % b                 # modulo (always non-negative for positive b)

# Avoid recursion limit for DFS
import sys
sys.setrecursionlimit(10**6)

# Swap
a, b = b, a           # no temp variable needed

# Ternary
val = x if condition else y

# Check if list is empty
if not arr: ...

# Copy
arr_copy = arr[:]          # shallow copy
import copy
deep = copy.deepcopy(obj)  # deep copy

# Large number mod
MOD = 10**9 + 7
(a + b) % MOD
pow(a, b, MOD)             # fast modpow

# Prefix sum
prefix = [0] * (n + 1)
for i in range(n):
    prefix[i+1] = prefix[i] + arr[i]
# range sum [l, r] (0-indexed, inclusive)
prefix[r+1] - prefix[l]

# Multiple return values
def func():
    return a, b
x, y = func()

# Unpacking
```

```
first, *rest = arr
*init, last = arr
a, b, *c = [1, 2, 3, 4, 5]
```

---

19. C++ STL → Python Quick Reference

C++	Python
`vector<int>`	`list`
`array<int, n>`	`list` or `[0]*n`
`string`	`str`
`stack<int>`	`list` (append/pop)
`queue<int>`	`collections.deque`
`deque<int>`	`collections.deque`
`priority_queue` (max)	`heapq` with negation
`priority_queue` (min)	`heapq`
`unordered_map`	`dict`
`map` (ordered)	`sortedcontainers.SortedDict`
`unordered_set`	`set`
`set` (ordered)	`sortedcontainers.SortedList`
`multiset`	`sortedcontainers.SortedList`
`pair<int,int>`	`tuple (a, b)`
`auto`	implicit
`lower_bound`	`bisect.bisect_left`
`upper_bound`	`bisect.bisect_right`
`sort()`	`arr.sort()` or `sorted()`
`reverse()`	`arr.reverse()` or `[::-1]`
`accumulate()`	`itertools.accumulate()`
`__gcd()`	`math.gcd()`
`INT_MAX`	`float('inf')`
`memset(arr, 0, ...)`	`[0] * n`
`fill()`	`[val] * n`

Python DSA Cheat Sheet — C++ to Python transition | Keep this close, refer often.